

Rapport SAE IA robot violet



L'objectif du projet est de programmer un robot AlphaI afin qu'il puisse se déplacer de manière autonome dans une arène sans heurter les murs. Le but final est d'atteindre la zone d'arrivée le plus rapidement possible grâce à des fonctions d'apprentissage et de décision codées en Python. Pour répondre à ces objectifs, nous utiliserons donc le réseau de neurones et le KNN. Nous nous sommes donc réparties les tâches de façon à ce que deux personnes travaillent sur le réseau de neurones et une personne sur le KNN.

Mise en oeuvre d'un apprentissage supervisé à l'aide d'un réseaux de neurone

On a décidé d'utiliser un réseau de neurones car on a pu remarquer que ça s'adapte mieux dans les conditions que le robot ne connaît pas et que pendant une course avec beaucoup d'autres robots, il y aurait beaucoup de cas jamais vus ou exploités.

La première décision que nous avons prise pour faire le réseau de neurones était de définir les entrées. On avait vu pendant les TP l'utilisation de l'écran séparé en 3 parties, mais nous avons trouvé que ça n'était pas suffisant car on voulait différencier le milieu de la droite et la gauche mais aussi de haut en bas car ça nous permettrait de mieux voir les virages puisqu'il y a une grosse différence entre le sol et les murs.

De plus, les 2 lignes de pixels en haut de la caméra voient au-dessus de la piste, et les pixels des coins de la ligne du bas sont toujours plus sombres que les autres donc nous avons décidé de les enlever. Nous avons également ajouté la distance pour pouvoir différencier des situations similaires.

Ensuite on a commencé le réseau de neurones, et on a donc dû définir la taille. Nous voulions une bonne capacité d'apprentissage, un réseau assez rapide, et pas trop compliqué à réaliser.

On a donc pris 24 entrées binaires pour les 6 zones de la caméra multipliées par les 4 couleurs possibles : blanc, noir, rouge, et autre plus une entrée pour la distance. Et en sortie on a essayé originellement avec trois sorties (avant, gauche, droite) mais finalement on a utilisé 4 sorties (avant, gauche, droite, arrière) pour pouvoir corriger les erreurs. Et on a ajouté une couche cachée de 32 neurones entièrement connectés.

De plus, pour les entraînements on a décidé de suivre le mur à gauche pour être sûr de ne pas se perdre, même si on perdait du temps. On a aussi différencié une base de données pour les qualifs et une pour la course car on s'est dit que si on entraîne le tour de qualif avec les autres il risquait d'être moins optimisé sur un tour.

Enfin on a rajouté des dernières améliorations comme le dash et des ajustements pour maximiser la vitesse et réduire le temps de déplacement de sorte à garder la même distance parcourue mais en moins de temps.

Mise en oeuvre d'un apprentissage supervisé à l'aide d'un algorithme KNN

En parallèle du programme en réseau de neurone, nous avons décidé de faire un deuxième code avec l'algorithme KNN.

Dans ce code, la stratégie était simple : avoir le moins de données possible pour obtenir le temps de réaction le plus court possible, et combiner l'image à la distance pour que l'algorithme puisse choisir le mouvement à effectuer. Pour limiter les données, nous avons décidé de choisir la résolution de caméra (16x12). Après plusieurs tests, nous nous sommes rendu compte que les 4 lignes de pixels du haut ne servaient pas, car elles captent principalement la couleur du plafond. Nous les avons donc retirées du traitement.

Dans cette idée d'optimisation, nous avons volontairement appris au robot un faible nombre de situations mémorisées lors de l'entraînement. C'est pourquoi nous avons paramétré notre algorithme des plus proches voisins avec $K=1$. En cherchant uniquement le voisin direct le plus similaire au lieu d'interroger plusieurs situations sur une base de données réduite, nous évitons les conflits de décision et garantissons un temps de calcul minimal pour le robot.

Ensuite, pour ne pas avoir à enregistrer les valeurs exactes de rouge, de vert et de bleu pour chaque pixel, nous avons écrit une fonction qui permet de déterminer si le pixel est blanc, rouge ou noir, puisque le robot devra agir différemment selon ces éléments. Au final, nous enregistrons une liste allégée constituée de valeurs simples (1, 2 ou 3 pour noir, rouge ou blanc), suivie de la valeur de la distance mesurée par le capteur ultrason.

De plus, lors de l'entraînement, nous n'avons pas seulement montré au robot la trajectoire parfaite mais nous l'avons aussi placé dans des situations où il est bloqué comme par exemple très proche d'un mur ou mal orienté pour lui apprendre les manœuvres de remise sur route avec les rotations sur place.

Et la dernière optimisation a été le "BOOST". Dès le lancement du mode course du robot, le robot avance le plus rapidement possible pendant 2 secondes tout droit pour prendre de l'avance par rapport aux autres robots.

Pour optimiser l'expérience utilisateur, le programme vérifie dès le lancement l'existence du dossier **memoire_robot_v2**. S'il n'y est pas, l'apprentissage démarre automatiquement. Si le dossier existe, une interface s'affiche, laissant 3 choix : 1 -> Le mode course, 2 -> Le mode apprentissage et 3 -> Suppression de l'apprentissage.

En mode apprentissage, cinq commandes intuitives (**z**, **q**, **d**, **a**, **e**) permettent de piloter le robot et d'enregistrer ses actions en temps réel. La touche **Échap** permet de sortir du mode apprentissage et de passer directement en mode course. Quant au choix 3, il permet de supprimer le document d'apprentissage avec une confirmation où il faut répondre par **o** ou **n** pour oui ou non.

Problèmes rencontrés

- bug du logiciel pour lancer le programme → solutions trouvées sur internet
- Perte de trajectoire du robot → solution : suivi du mur gauche (rouge)
- Prise de décisions incorrectes → simuler les cas d'erreur spécifiques pour lui apprendre à ne pas les refaire

Nous avons appris et constaté la limite de l'algorithme KNN par rapport au réseau de neurones car il utilisait les cas les plus proches et les plus récents alors que le réseau de neurones s'adaptait au fur et à mesure de l'apprentissage ce qui permettait de continuer la course même si le circuit changeait. On a aussi appris à utiliser des couches d'entrées et de sortie en exploitant les pixels utilisés pour optimiser la visualisation des couleurs rgb. Pour le robot, nous aurions peut-être pu lui apprendre plus de cas différents et lui ajouter des vitesses de déplacement spécifiques à chaque angle et distance par rapport aux murs et aux robots pour être plus fluide pendant la course.